Target Specification, Not Model Capability, Explains Most Unit-Test Generation Failures at Medium Complexity

Nguyễn Tùng Ân, Đoàn Thị Thu Kim, Sơn Hải, Trương Hoàng Phúc, and Trần Xuân Lộc

FPT University, Vietnam

Abstract. **Keywords:** Unit test generation · Large language models · Mutation testing · Cyclomatic complexity · Measurement validity

1 intro

2 related

3 Methodology

3.1 Dataset Construction

We mine functions from ten open-source repositories pinned at fixed commits (eight Java projects: `commons-cli`, `commons-csv`, `commons-collections`, `commons-math`, `gson`, `jsoup`, `joda-time`, `jfreechart`; two Python projects: `flask`, `requests`). Cyclomatic complexity is measured with Lizard 1.23.0 and we keep the medium band $5 \leq CC \leq 10$, matching the complexity range studied in our earlier work.

Complexity alone, however, does not make a function a valid test target. A study that samples on complexity but ignores *testability* measures two things at once: how well a generator writes tests, and how often the sampled target could have been tested at all. We therefore apply five additional criteria, each fixed before any test was generated and each justified by whether *any* tool — an LLM, a search-based generator, or a human — could reach the target through the public API.

F1 — **Non-trivial body.** $nloc \geq 3$. Removes stubs, abstract declarations and interface signatures, which contain no branches to cover.

F2 — **Public or exported.** Java: `public` only. Python: names not prefixed with an underscore. A `private` method cannot be invoked from an external test class by ordinary means; reflection can reach it, but then the compiler no longer checks any name in the test (Section 3.3), so compilation ceases to be evidence of anything.

Table 1. Sampling funnel. Each row reports how many candidates survive the criterion.

Stage	Criterion	Remaining
F0	$5 \leq CC \leq 10$ (Lizard)	1,848
F1	non-trivial body	1,848
F3	unambiguous name	753
F2	public / exported	474
F5	dynamically resolvable (Python)	64
	final pool	Java 406, Python 64
	sampled	Java 60, Python 60

F3 — Unambiguous name. Java: the method name occurs exactly once in its declaring class. Python: the function name occurs exactly once in its module. An overloaded name cannot be specified unambiguously in a prompt, and the compiler rejects the resulting call as ambiguous.

F4 — Verified declaring class. The enclosing class is resolved with a brace-depth stack that *pops* on block exit. Taking "the nearest enclosing class declaration" without popping attributes a method to whichever nested class was declared most recently, which is wrong whenever the target follows a nested type in the same file.

F5 — Dynamically resolvable (Python). The module is imported, the qualified name is walked with `getattr`, and the result is confirmed callable. Syntactic inference is not evidence that a target can be reached at run time.

Table 1 reports the full funnel. We report every stage rather than only the final count, because the attrition is itself a finding: of 1,848 functions in the complexity band, only 474 (26 %) are simultaneously public and unambiguous. Roughly three quarters of medium-complexity functions in these projects are not valid targets for an external test generator.

From the surviving pool we sample 60 Java and 60 Python functions, balanced across repositories (Java: 3–9 functions per project; Python: 32 from `requests`, 28 from `flask`). Complexity is concentrated at the lower end of the band (CC=5: 56 functions; $CC \geq 8$: 23), and the median function is 17 lines. Of the Python targets, 34 are instance methods and 26 are module-level functions. Every one of the 120 targets was confirmed resolvable at run time before generation began.

3.2 Test Generation

All tests are produced by `gpt-4o-mini-2024-07-18` at `temperature` = 0, `top_p` = 1.0, `max_tokens` = 2048, in a **single API call per function** with no feedback loop and no retrieval.

We compare two *one-shot* prompt conditions. Both contain exactly one worked exemplar (a trivial `add` function with its test), so both are one-shot in the standard sense, and both issue one call. They differ in exactly one respect:

V1 — baseline. The prompt names the target function and shows its source. This is the prompt used in our earlier study.
V2 — target-specified. Identical to V1, with one additional block inserted before the task: the fully qualified target name, its exact signature, its declaring class, and — for instance methods — a constructor call that has been *executed* and confirmed to build the receiver successfully.

Holding the exemplar fixed matters. An earlier draft of the enriched prompt was rewritten from scratch and, in the process, dropped the exemplar; it therefore differed from V1 in two respects at once (no exemplar, plus target metadata) and no causal attribution was possible from it. We report that variant only as a secondary observation and label it zero-shot, since it is one.

3.3 Measurement: Four Tiers

A single "valid / invalid" flag conflates outcomes that differ in kind, so we report four nested tiers. Each answers a different question, and — importantly — the first two can be satisfied by a test that verifies nothing.

T1 — Compiles / collects. The test file is syntactically and type-correct.
T2 — At least one green test. At least one test passes against the unmodified source.
T3 — Reaches the target. Branch coverage > 0 within the target's line range $[start, end]$.
T4 — Detects faults. Mutation score > 0 within the same line range.

Only T4 supports the claim that a suite *tests* the target. We show that T1 and T2 are both reachable without testing anything:

- **T1 under reflection.** Reflection turns every name into a string, so the compiler stops checking it. A call to `getDeclaredMethod("aNameThatDoesNotExist")` compiles with return code 0 and fails only at run time. Any compilation-based validity rate is therefore uninformative for suites that use reflection.
- **T2 under an empty suite.** One search-based suite in our corpus consists of four lines: a single test that assigns a string literal, importing nothing and calling nothing. It passes, and a green-test criterion records it as a success.

Branch coverage is measured with `coverage.py` (Python) and by executing the generated JUnit suites through the JUnit Platform launcher (Java), in both cases attributed to the target's line range rather than to the whole file. Mutation analysis applies the same operator set to both languages — arithmetic ($+/-$, $\times/\div$), relational ($> / \geq$, $< / \leq$), equality ($==/\neq$), boolean ($\wedge/\vee$), boolean constant inversion, and numeric constant increment — capped at 20 mutants per function, restricted to the target's line range.

Two procedural safeguards are worth stating because their absence silently corrupts results. First, mutation requires overwriting the source; we mutate a *copy* of the source tree placed ahead of the original on the import path, so

that concurrent measurements reading the same tree are unaffected. Second, tool versions must be matched across families: JUnit 5 pairs Jupiter 5.x with Platform 1.x, and mixing a Jupiter 6 artifact with JUnit 5-style tests makes the launcher discover zero tests while exiting successfully — a failure that reports as a legitimate zero.

3.4 Baselines

Each baseline generates per class (Java) or per module (Python), matching how the tools operate; results are then attributed to individual functions by line range, exactly as for the LLM suites.

EvoSuite 1.2.0 , 60 s per class, run under JDK 11. EvoSuite performs deep reflection into JDK internals, which the module system blocks from Java 9 onward, and its master process launches its client from `JAVA_HOME` — so both must point at the same JDK, or the client dies while the master still exits successfully. Because the projects were compiled with JDK 17, we recompiled sources to Java 11 bytecode into a separate output directory, leaving the original build untouched.

Randoop 4.3.3 , 60 s per class, JDK 17.

Pynguin 0.45.0 , 90 s per module, under **two configurations reported side by side**. In its default configuration Pynguin serialises module state to a worker process with `dill` and aborts on C-extension type objects that cannot be resolved by name, producing no output at all for several modules. We therefore also run it with the master–worker split disabled. Reporting both separates "the tool scored low" from "the tool never ran".

3.5 Statistical Analysis

Conditions are compared with the paired Wilcoxon signed-rank test, matching on function identifier, at $\alpha = 0.05$, with matched-pairs rank-biserial correlation as effect size. Java and Python are analysed separately throughout: the toolchains differ, and Java enforces access control while Python does not, so their failure modes are not commensurable. Every rate is reported over the 60 functions of its own language.

Results are given at two levels in parallel: *all* ($n = 60$ per language, with unmeasurable cases scored 0) and *effective* (the subset with a non-zero value). The *all* level is reported even where the *effective* level is more favourable.

All configuration decisions above — criteria, tool versions, budgets, the two-branch Pynguin design, and the hypotheses — were recorded in a pre-registration committed before any baseline was executed.

4 results

5 discussion

6 Threats to Validity

6.1 Construct Validity

What a validity flag measures. The central construct threat in this area is that the usual "valid suite" indicator does not measure what its name suggests, and fails silently when it does not. We encountered four distinct instances, three of them in our own instrumentation.

First, a compilation flag inferred from a coverage report measures only whether the class appears in that report. Coverage tools enumerate every class in a project, including classes that were never loaded; such a class yields zero covered branches, which the parser then records as a successfully compiled function with zero coverage. Second, a criterion of "at least one test executed" counts a file in which every test fails. Third, reflection removes the compiler's ability to check names, so a compilation-based rate is uninformative for any suite that uses it. Fourth, mismatched tool versions can make a test launcher discover zero tests while exiting successfully.

None of these produce an error. Each returns a plausible number. We therefore report the four nested tiers of Section 3.3 rather than any single flag, and treat only T4 — fault detection within the target's line range — as evidence that a suite tests its target.

Mutation operators. Our operator set is deliberately small and identical across both languages, so that the two are measured with the same instrument. It is weaker than a production mutation framework, so absolute mutation scores are not comparable with studies using PIT or `mutmut`; only within-study comparisons are meaningful. We also cap mutants at 20 per function, which bounds the resolution of the score for larger functions.

6.2 Internal Validity

Sampling. Our own miner mis-attributed one Java target: `LocalTime.getField` is declared `protected`, but the parser matched a same-named declaration in a nested class and recorded it as public, so it passed the public-only filter. An audit against the real declaration line found this to be the only such case in 60 Java functions; we exclude it from the analysis. We record it here because it is precisely the class-attribution error this study is about, reproduced in our own tooling — knowing about a failure mode does not confer immunity to it.

Concurrency. Mutation analysis overwrites source files. Running it directly on a shared source tree corrupts any concurrent measurement reading that tree, silently. We mutate an isolated copy placed ahead of the original on the import path, and verify after each run that the original tree is unmodified.

6.3 External Validity

Source exhaustion in Python. After all testability criteria, only 64 Python functions in the two projects qualify, of which 34 already appeared in our earlier dataset. A strictly held-out Python replication is therefore limited to roughly 30 functions without adding new repositories. Our Python sample is also drawn from two web-oriented libraries and may not generalise to other domains.

Complexity distribution. Although the band is $5 \leq CC \leq 10$, the sample concentrates at the lower end (56 of 120 functions have $CC = 5$). Conclusions about the upper half of the band rest on fewer observations.

Model and configuration. A single model at a single temperature was used. We do not claim the findings transfer to other models, and the deterministic setting means we do not estimate run-to-run variance.

6.4 Conclusion Validity

Baseline configuration. Three limitations constrain how far the baseline comparison can be pushed. EvoSuite was run with a classpath containing only the module's own compiled classes: with the full dependency classpath its bytecode reader aborts in a way that produces no output while reporting success, and we were unable to identify the specific offending artifact. EvoSuite may therefore be weaker here than in a fully configured environment. Seven Java targets in one multi-module project could not be recompiled to Java 11 bytecode and fall outside EvoSuite's coverage. Finally, our Randoop version differs from the one used in our earlier study, so Randoop numbers are compared only within this study.

Pre-registration and one deviation. Research questions, hypotheses, tool versions, budgets and the two-branch Pynguin design were committed before any baseline was run. One condition was added afterwards: on inspection, the enriched prompt we had planned to use turned out to lack the worked exemplar, making it zero-shot rather than one-shot and differing from the baseline in two respects at once. We added a corrected condition that holds the exemplar fixed and varies only the target specification, recorded the change in the pre-registration with its date, and left the original hypothesis unaltered. The uncorrected variant is reported only as a secondary observation.

7 conclusion